

Project Executable Files & Execution Guide

Date	07 July 2026
Project Name	CaptionAI: AI Powered Social Media Caption Generator
Maximum Marks	5 Marks

Project Executable Files & Execution Guide:

CaptionAI is a Flask-based web application that generates AI-powered social media captions, hashtags, and call-to-action (CTA) suggestions. The application uses the Groq API for AI-powered content generation and SQLite for storing user details and caption history. This guide documents the project repository, folder structure, and the steps required to build and run the application locally.

Step 1: Project Links & Access

Add the links below before submission/evaluation.

Resource Type	URL / Access Link
Source Code Repository (GitHub)	[Add your GitHub repository link here]
Live Deployed Application (Render)	[Add your Render deployment link here]
Demo Video Link (Optional but recommended)	[Add a link to a YouTube or Drive video demonstrating the project]

Step 2: Key Executable Files & Folder Structure

S.No	Folder / File Name	Description & Purpose
1	app.py	Main Flask application file; entry point that defines all application routes and logic.
2	database.py	Handles the SQLite database connection and operations.
3	caption_history.db	SQLite database file storing user details and generated caption history.
4	templates/	Contains the HTML pages: index.html, login.html, register.html, dashboard.html, and history.html.
5	static/css/, static/js/, static/images/	Static assets folder containing stylesheets, JavaScript files, and images/icons.
6	requirements.txt	Lists all Python dependencies required to run the project.
7	.env	Stores the Groq API key and other environment variables securely (not committed to version control).
8	README.md	Project documentation describing setup and usage.

Step 3: Local Setup & Execution Instructions

Step	Action / Terminal Command	Expected Outcome / Notes
1	<code>git clone {repository_url}</code>	Downloads the project to the local machine.
2	<code>cd CaptionAI</code>	Opens the project folder (open it in Visual Studio Code for editing).
3	<code>python -m venv venv</code> <code>venv\Scripts\activate</code> (Windows) <code>source venv/bin/activate</code> (macOS/Linux)	Creates and activates a Python virtual environment.
4	<code>pip install -r requirements.txt</code>	Installs all required dependencies (Flask, Requests, Werkzeug, Jinja2, python-dotenv, etc.).
5	Create a <code>.env</code> file and add: <code>GROQ_API_KEY=your_api_key_here</code>	Securely stores the Groq API key needed for AI caption generation.
6	<code>python app.py</code>	Launches the Flask application.
7	Open <code>http://127.0.0.1:5000</code> in a web browser	Opens the CaptionAI interface in the browser (Google Chrome or Microsoft Edge recommended).

Software Requirements

Operating System	Windows 10/11, Linux, or macOS
Programming Language	Python 3.10 or above
Framework	Flask
Database	SQLite
Code Editor	Visual Studio Code
Web Browser	Google Chrome or Microsoft Edge

Required Python Libraries

- Flask
- Requests
- SQLite3
- Werkzeug
- Jinja2
- python-dotenv

Application Workflow

- 1 Launch the Flask application.
- 2 Register a new account.

- 3 Log in using valid credentials.
- 4 Enter the content prompt.
- 5 Select the desired social media platform.
- 6 Choose the preferred writing tone.
- 7 Click the Generate Caption button.
- 8 View the generated caption, hashtags, and CTA suggestions.
- 9 Copy, download, or save the generated content.
- 10 Access previous captions through the History page.

Output

- AI-powered social media captions.
- Relevant hashtags.
- Call-to-action (CTA) suggestions.
- Caption history.
- Downloadable caption content.

Error Handling

- Displays validation messages for empty input fields.
- Handles invalid login credentials.
- Displays an error message if the Groq API is unavailable.
- Prevents application crashes through exception handling.

Deployment

Development Environment	Localhost using Flask
Testing	Ngrok
Production Deployment	Render

Conclusion

The CaptionAI application can be executed using Python and Flask with minimal setup. The modular folder structure, SQLite database, and Groq API integration make the project easy to run, maintain, and extend. Following the execution steps ensures successful installation and smooth operation of the application.